

Lógica Computacional



Prof. Esp. Wesley Neves

10



AULA 10 – Introdução a Python





Conteúdo

Introdução ao Python e à programação informática

Neste módulo, aprenderá sobre:

- os fundamentos da programação informática, ou seja, como o computador funciona, como o programa é executado, como a linguagem de programação é definida e construída;
- a diferença entre compilação e interpretação;
- o que é o Python, como se posiciona entre outras linguagens de programação, e o que distingue as diferentes versões de Python.



UNIDESC

Module 1:

Introduction to Python
and computer programming

Python Essentials 1

1

2

3

4





Como funciona um programa de computador?

Este curso tem como objetivo mostrar o que é a linguagem Python e para o que é utilizada Vamos começar a partir do básico.

Um programa ⁺ torna um computador utilizável. Sem um programa, um computador, mesmo o mais poderoso, nada mais é do que um objeto. Da mesma forma, sem um pianista, um piano não é mais do que uma caixa de madeira.



Os computadores são capazes de executar tarefas muito complexas, mas essa capacidade não lhes é inata. A natureza de um computador é bastante diferente.

Ele só pode executar operações extremamente simples, por exemplo, um computador não pode avaliar o valor de uma função matemática complicada por si só, embora isto não esteja fora do âmbito das possibilidades num futuro próximo.

Computadores contemporâneos só podem avaliar os resultados de operações muito fundamentais, como adicionar ou dividir, mas podem fazê-lo muito

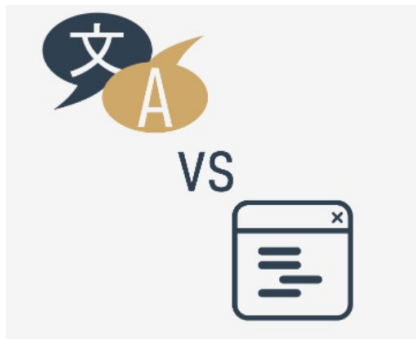
rapidamente, e podem repetir estas ações virtualmente um qualquer número de vezes.



Linguagens naturais vs. linguagens de programação

Uma linguagem é um meio (e uma ferramenta) para expressar e registrar pensamentos. Há muitas linguagens ao nosso redor. Algumas delas não requerem nem a fala nem a escrita, como a linguagem corporal; é possível expressar os seus sentimentos mais profundos com muita precisão sem dizer uma palavra.

Outra linguagem que usa diariamente é a sua língua materna, que usa para manifestar a sua vontade e para pensar na realidade. Os computadores também têm a sua própria linguagem, chamada linguagem de máquina, que é muito rudimentar.





O que faz uma linguagem?

Podemos dizer que cada linguagem (de máquina ou natural, não importa) é constituída pelos seguintes elementos:

- um **alfabeto**: um conjunto de símbolos utilizados para construir palavras de uma determinada linguagem (por exemplo, o alfabeto latino para inglês, o alfabeto cirílico para russo, o Kanji para japonês, etc.)
- um **lexis**: (ou seja, um dicionário) um conjunto de palavras que a linguagem oferece aos seus utilizadores (por exemplo, a palavra "computador" vem do dicionário de língua inglesa, enquanto que "cmoptrue" não; a palavra "chat" está presente tanto nos dicionários de inglês como de francês, mas os seus significados são diferentes)
- uma **sintaxe**: um conjunto de regras (formais ou informais, escritas ou sentidas intuitivamente) utilizadas para determinar se uma determinada sequência de palavras forma uma frase válida (por exemplo, "Eu sou uma pitão" é uma frase sintaticamente correta, enquanto "Eu uma pitão sou" não é)
- **semântica**: um conjunto de regras que determinam se uma determinada frase faz sentido (por exemplo, "Comi um donut" faz sentido, mas "Um donut comeu-me" não faz)



Linguagem de computação

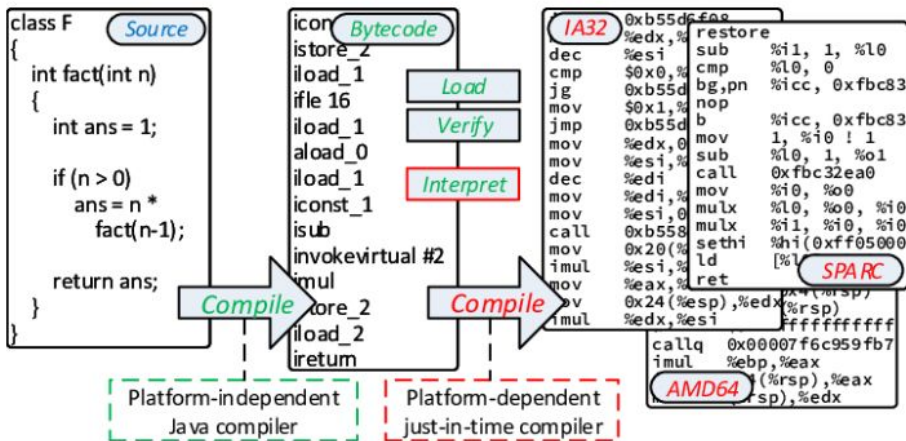
O IL é, de facto, **o alfabeto de uma linguagem de máquina**. Este é o conjunto mais simples e primário de símbolos que podemos utilizar para dar comandos a um computador. É a língua materna do computador.

Figure 1 - uploaded by [Michael Orlov](#)

Content may be subject to copyright.

Download

View publication





Compilação vs. interpretação

Há duas formas diferentes de **transformar um programa de uma linguagem de programação de alto nível em linguagem de máquina**:

COMPILAÇÃO - o source program é traduzido uma vez (no entanto, este ato deve ser repetido sempre que modificar o source code) obtendo um ficheiro (por exemplo, um ficheiro .exe se o código se destinar a ser executado no MS Windows) contendo o machine code; agora pode distribuir o ficheiro por todo o mundo; o programa que executa esta tradução chama-se compilador ou tradutor;

INTERPRETAÇÃO - você (ou qualquer utilizador do código) pode traduzir o source program cada vez que este tem de ser executado; o programa que executa este tipo de transformação chama-se intérprete, pois interpreta o código cada vez que se pretende executá-lo; também significa que não pode simplesmente distribuir o source code tal como está, porque o utilizador final também precisa do intérprete para o executar.



Compilação vs. interpretação

O que é que o intérprete realmente faz?

Vamos assumir mais uma vez que escreveu um programa. Agora, existe como um **ficheiro de computador**: um programa de computador é na realidade um pedaço de texto, por isso o source code é normalmente colocado em **ficheiros de texto**.

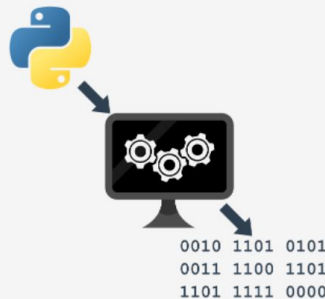
Nota: tem de ser **texto puro**, sem quaisquer decorações como diferentes fontes, cores, imagens embutidas ou outros suportes. Agora tem de invocar o intérprete e deixá-lo ler o seu source file.

O intérprete lê o source code de uma forma que é comum na cultura ocidental: de cima para baixo e da esquerda para a direita. Há algumas exceções - elas serão abordadas mais tarde no curso.

Em primeiro lugar, o intérprete verifica se todas as linhas subsequentes estão corretas (utilizando os quatro aspetos abordados anteriormente).

Se o compilador encontrar um erro, termina o seu trabalho imediatamente. O único resultado, neste caso, é uma **mensagem de erro**.

O intérprete informá-lo-á onde se encontra o erro e o que o causou. No entanto, estas mensagens podem ser enganadoras, uma vez que o intérprete não é capaz de seguir exatamente as suas intenções, e pode detetar erros a alguma distância das suas verdadeiras causas.





Compilação vs. interpretação

COMPILAÇÃO

VANTAGENS

- a execução do código traduzido é geralmente mais rápida;
- apenas o utilizador tem de ter o compilador - o end-user (utilizador final) pode usar o código sem ele;
- o código traduzido é armazenado utilizando linguagem de máquina - como é muito difícil de entender, as suas próprias invenções e truques de programação provavelmente permanecerão segredo.

DESVANTAGENS

- a compilação em si pode ser um processo muito demorado - pode não ser capaz de executar o seu código imediatamente após qualquer alteração;
- tem de ter tantos compiladores quanto plataformas de hardware em que queira que o seu código seja executado.



Compilação vs. interpretação

VANTAGENS

- pode executar o código assim que o concluir - não há fases adicionais de tradução;
- o código é armazenado usando linguagem de programação, não de máquina - isto significa que pode ser executado em computadores utilizando diferentes linguagens de máquina; não se compila o código separadamente para cada arquitetura diferente.

INTERPRETAÇÃO

DESVANTAGENS

- não espere que a interpretação aumente o seu código para alta velocidade - o seu código irá partilhar o poder do computador com o intérprete, pelo que não pode ser realmente rápido;
- tanto você como o end-user têm de ter o intérprete para executar o seu código.



UNIDESC

Nascimento Python

Guido
van
Rossum



O que é o Python?

O Python é uma linguagem de programação de grande utilização, interpretada, orientada a objetos, e de alto nível com semântica dinâmica, utilizada para programação de uso geral.

E embora possa conhecer o python (pitão) como uma grande cobra, o nome da linguagem de programação **Python vem de uma antiga série de comédia televisiva da BBC chamada Monty Python's Flying Circus**



Objetivos do Python

Em 1999, Guido van Rossum definiu os seus objetivos para o Python:

- uma linguagem **fácil e intuitiva**, tão poderosa como a dos principais concorrentes;
- de **open source**, para que qualquer pessoa possa contribuir para o seu desenvolvimento;
- código que seja tão **compreensível** como o inglês simples;
- **adequado para tarefas quotidianas**, permitindo tempos de desenvolvimento curtos.

Cerca de 20 anos mais tarde, é evidente que todas estas intenções foram cumpridas. Algumas fontes dizem que o Python é a linguagem de programação mais popular no mundo, enquanto outras afirmam que é a terceira ou a quinta.



O que torna o Python especial?

Como é que os programadores, jovens e velhos, experientes e novatos, querem utilizá-lo? Como aconteceu que grandes empresas adotassem o Python e implementassem os seus principais produtos utilizando-o?

Há muitas razões - já enumerámos algumas delas, mas vamos enumerá-las novamente de uma forma mais prática:

- **é fácil de aprender** - o tempo necessário para aprender Python é menor do que para muitas outras linguagens; isto significa que é possível iniciar a programação em si mais rapidamente;
- **é fácil de ensinar** - a carga de trabalho de ensino é menor do que a necessária para outras linguagens; isto significa que o professor pode colocar mais ênfase em técnicas de programação gerais (independentes da linguagem), não desperdiçando energia em truques exóticos, estranhas exceções e regras incompreensíveis;
- **é fácil de usar** para escrever novo software - é muitas vezes possível escrever código mais rapidamente quando se usa Python;
- **é fácil de compreender** - é também frequentemente mais fácil e rápido de compreender o código de outra pessoa se for escrito em Python;
- **é fácil de obter, instalar e implementar** - o Python é gratuito, aberto e multiplataforma; nem todas as linguagens se podem gabar disso.



Porque não Python?

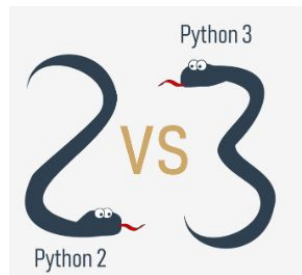
Apesar da popularidade crescente de Python, ainda existem alguns nichos onde o Python está ausente, ou raramente é visto:

- **programação de baixo nível** (por vezes chamada programação "close to metal"): se quiser implementar um condutor ou motor gráfico extremamente eficaz, não utilizaria Python;
- **aplicações para dispositivos móveis**: embora este território ainda esteja à espera de ser conquistado pelo Python, é muito provável que um dia tal venha a acontecer.



UNIDESC

Nascimento Python



Há mais do que um Python

Existem dois tipos principais de Python, chamados Python 2 e Python 3.

O Python 2 é uma versão mais antiga do Python original. Desde então o seu desenvolvimento tem sido intencionalmente parado, embora isso não signifique que não hajam atualizações. Pelo contrário, as atualizações são emitidas regularmente, mas não se destinam a modificar a linguagem de forma significativa. Preferem corrigir quaisquer bugs recém-descobertos e falhas de segurança. O caminho de desenvolvimento de Python 2 já chegou a um beco sem saída, mas o Python 2 em si ainda está muito vivo.

Python 3 é a versão mais recente (para ser mais preciso, a atual versão) da linguagem. Está a percorrer o seu próprio caminho de evolução, criando os seus próprios padrões e hábitos.



Cython

Outro membro da família Python é o **Cython**.



O Cython é uma das várias soluções possíveis para a mais dolorosa das características de Python - a falta de eficiência. Grandes e complexos cálculos matemáticos podem ser facilmente codificados em Python (muito mais facilmente do que em "C" ou qualquer outra linguagem tradicional), mas a execução do código resultante pode ser extremamente demorada.

Como são conciliadas estas duas contradições? Uma solução é escrever as suas ideias matemáticas usando Python, e quando estiver absolutamente seguro de que o seu código está correto e produz resultados válidos, pode traduzi-lo para "C". Certamente, o "C" correrá muito mais rápido do que Python puro.

É isto que o Cython pretende fazer - traduzir automaticamente o código Python (limpo e claro, mas não demasiado rápido) em código "C" (complicado e falador, mas ágil).



Nascimento Python

Jython

Outra versão do Python é chamada **Jython**.

“J” é para “Java”. Imagine um Python escrito em Java em vez de C. Isto é útil, por exemplo, se desenvolver sistemas grandes e complexos escritos inteiramente em Java, e quiser acrescentar alguma flexibilidade Python a eles. O CPython tradicional pode ser difícil de integrar em tal ambiente, já que C e Java vivem em mundos completamente diferentes e não partilham muitas ideias comuns.

Jython pode comunicar com a infra-estrutura Java existente de forma mais eficaz. É por isso que alguns projetos o consideram utilizável e necessário.

Nota: a atual implementação do Jython segue as normas do Python 2. Até ao momento, não há Jython em conformidade com Python 3.



Nascimento Python

PyPy e RPython

Dê uma vista de olhos ao logotipo em baixo. É um rébus. Consegue resolvê-lo?



É um logótipo do **PyPy** - um Python dentro de um Python. Por outras palavras, representa um ambiente Python escrito em linguagem Python, chamado **RPython** (Restricted Python). Na verdade, é um subconjunto de Python.

O source code de PyPy não é executado na forma de interpretação, mas sim traduzido para a linguagem de programação C e depois

executado separadamente.

Isto é útil porque se quiser testar qualquer nova funcionalidade que possa ser (mas não tem de ser) introduzida na implementação do Python convencional, é mais fácil verificá-la com o PyPy do que com o CPython. É por isto que o PyPy é antes uma ferramenta para pessoas que desenvolvem Python, do que para o resto dos utilizadores.

Isto não torna o PyPy menos importante ou menos sério do que o CPython, é claro.

Além disso, o PyPy é compatível com a linguagem do Python 3.

Existem muitos mais Pythons diferentes no mundo. Encontrá-los-á se procurar, mas **este curso irá concentrar-se no CPython.**



UNIDESC

Nascimento Python

<https://www.python.org/downloads/>
<https://pypi.org/>

```
C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.9_3.9.3312.0_x64__qbz5n2kfra8p0\python3.9.exe
```

```
Python 3.9.12 (tags/v3.9.12:b28265d, Mar 23 2022, 23:52:46) [MSC v.1929 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> 1+1
```

```
2
```

```
>>>
```




Descarregar e instalar o Python

Como o browser diz ao site onde entrou o sistema operativo que utiliza, o único passo que tem de dar é clicar na versão Python apropriada que deseja.

Neste caso, selecione Python 3. O site oferece sempre a versão mais recente do mesmo.

Se for um **utilizador do Windows**, inicie o ficheiro .exe descarregado e siga todos os passos.

Deixe as configurações padrão que o instalador sugere por agora, com uma exceção - veja a caixa de verificação chamada **Add Python 3.x to PATH** e verifique-a.

Isto tornará as coisas mais fáceis.

Se for um **utilizador MacOS**, uma versão do Python 2 pode já ter sido pré-instalada no seu computador, mas como vamos trabalhar com o Python 3, ainda assim terá de descarregar e instalar o ficheiro .pkg relevante a partir do site Python.



Iniciar o seu trabalho com Python

Agora que tem o Python 3 instalado, é altura de verificar se funciona, e fazer o primeiro uso do mesmo.

Este será um procedimento muito simples, mas deve ser o suficiente para o convencer de que o ambiente Python é completo e funcional.

Existem muitas formas de utilizar o Python, especialmente se vier a ser um programador Python.

Para começar o seu trabalho, precisa das seguintes ferramentas:

- um **editor** que o irá apoiar na escrita do código (deve ter algumas características especiais, não disponíveis em ferramentas simples); este editor dedicado dar-lhe-á mais do que o equipamento padrão do sistema operativo;
- uma **consola** na qual pode correr o seu código recém-escrito e pará-lo à força quando ficar fora de controlo;
- uma ferramenta chamada um **debugger**, capaz de correr o seu código passo a passo e que lhe permite inspecioná-lo em cada momento da execução.

Para além dos seus muitos componentes úteis, a instalação padrão de Python 3 contém uma aplicação muito simples mas extremamente útil chamada IDLE.

IDLE é um acrónimo: Integrated Development and Learning Environment.



```
IDLE Shell 3.9.12
File Edit Shell Debug Options Window Help
Python 3.9.12 (tags/v3.9.12:b28265d, Mar 23 2022, 23:52:46) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> |
Ln: 3 Col: 4
```

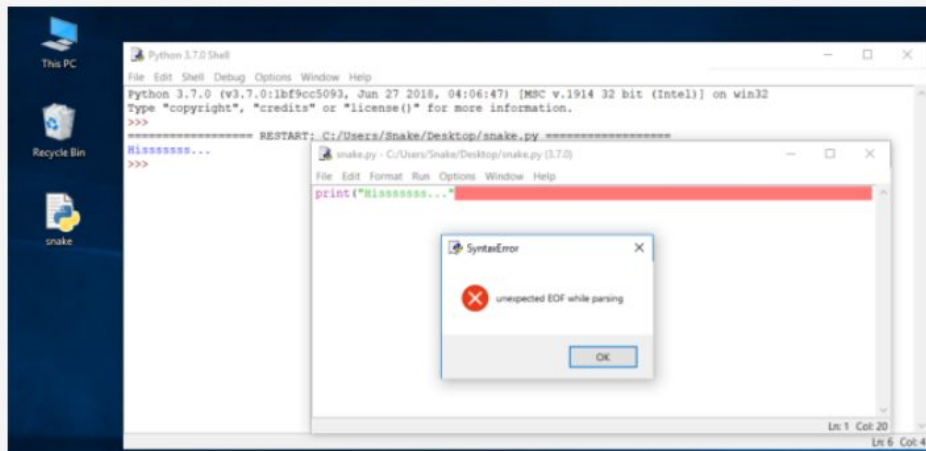


Como estragar e corrigir o seu código

Observe cuidadosamente todas as janelas.

Uma nova janela – diz que o intérprete encontrou um EOF (*end-of-file*) embora (na sua opinião) o código deva conter mais algum texto.

A janela do editor mostra claramente onde isto aconteceu.





```
1
2 ispar(2)
3
4 def isPar(numero):
5     return numero % 2 ==0;
6
7
8
9
```

Console >_

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    ispar(2)
NameError: name 'ispar' is not defined
```



```
▶ 📄 📄 🔗 ⬇ 📁 ↺ 🏠 ⚙
1
2 isPar(2)
3
4 def isPar(numero):
5     return numero % 2 ==0;
6
7
```

Console >_

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    isPar(2)
NameError: name 'isPar' is not defined
```




```
▶ 📄 📄 🔗 ⬇ 📁 ↺ 🏠 ⚙
```

```
1  
2 isPar(2)  
3  
4 def isPar(numero):  
5     return numero % 2 ==0;  
6  
7
```

Console >_

```
Traceback (most recent call last):  
  File "main.py", line 2, in <module>  
    isPar(2)  
NameError: name 'isPar' is not defined
```



```
1  
2  
3  
4 ▾ def isPar(numero):  
5     return numero % 2 == 0;  
6  
7  
8 print(isPar(2))
```

Console >_

True



A mensagem (a **vermelho**) mostra (nas linhas subsequentes):

- o **traceback** (que é o caminho que o código percorre através de diferentes partes do programa - pode ignorá-lo por agora, uma vez que está vazio num código tão simples);
- a **localização do erro** (o nome do ficheiro contendo o erro, o número da linha e o nome do módulo); nota: o número pode ser enganador, uma vez que o Python normalmente mostra o local onde primeiro se notam os efeitos do erro, não necessariamente o erro em si;
- o **conteúdo da linha errada**; nota: a janela do editor IDLE não mostra os números das linhas, mas mostra a localização atual do cursor no canto inferior direito; use-a para localizar a linha errada num source code longo;
- o **nome do erro** e uma breve explicação.



Como estragar e corrigir o seu código

Deve ter notado que a mensagem de erro gerada para o erro anterior é bastante diferente da primeira.

The screenshot shows a Windows desktop with icons for 'This PC', 'Recycle Bin', and a file named 'snake'. A 'Python 3.7.0 Shell' window is open, displaying the following text:

```
Python 3.7.0 (tags/v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Snake/Desktop/snake.py =====
Missssss...
>>>
===== RESTART: C:/Users/Snake/Desktop/snake.py =====
Traceback (most recent call last):
  File "C:/Users/Snake/Desktop/snake.py", line 1, in <module>
    prin("Missssss...")
NameError: name 'prin' is not defined
>>> |
```

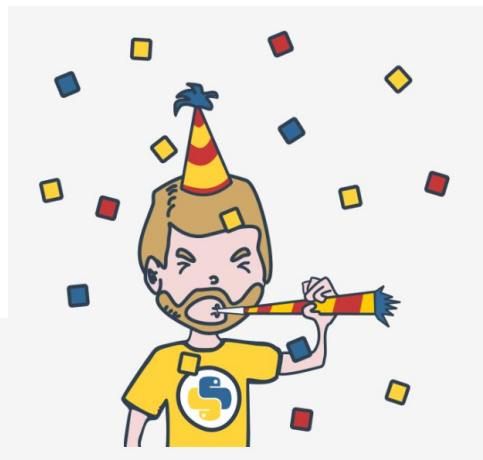
The error message is a `NameError` indicating that the variable `prin` is not defined, which is a misspelling of `print`. The status bar at the bottom right of the window shows 'lin 12 Col 4'.



Parabéns! **Concluiu o Módulo 1.**

Muito bem! Chegou ao fim do Módulo 1 e completou um marco importante na sua educação em programação Python. Aqui está um breve resumo dos objetivos que abordou e com os quais se familiarizou no Módulo 1:

- os fundamentos da programação informática, ou seja, como o computador funciona, como o programa é executado, como a linguagem de programação é definida e construída;
- A diferença entre compilação e interpretação;
- a informação básica sobre Python e como se posiciona entre outras linguagens de programação, e o que distingue as suas diferentes versões;
- os recursos de estudo e os diferentes tipos de interfaces que irá utilizar no curso.





UNIDESC

Final Modulo 1

Your score: **9/10**

90%

Congratulations, you've passed the quiz!

SECTION ANALYSIS

PE1 -- Module 1 Quiz	90%
----------------------	-----

[Retake Test](#)

[Review Test](#)